

第45章 RS-485 通讯实验

本章参考资料：《STM32F4xx 中文参考手册》USART 章节。

学习本章时，配合本书前面的《USART—串口通讯》及《CAN—通讯实验》章节进行对比学习，效果更佳。

关于实验板中使用的 MAX485 收发器资料可查阅《MAX485》规格书了解。

45.1 RS-485 通讯协议简介

与 CAN 类似，RS-485 是一种工业控制环境中常用的通讯协议，它具有抗干扰能力强、传输距离远的优点。RS-485 通讯协议由 RS-232 协议改进而来，协议层不变，只是改进了物理层，因而保留了串口通讯协议应用简单的特点。

45.1.1 RS-485 的物理层

从《CAN—通讯实验》章节中了解到，差分信号线具有很强的抗干扰能力，特别适用于电磁环境复杂的工业控制环境中，RS-485 协议主要是把 RS-232 的信号改进成差分信号，从而大大提高了抗干扰特性，它的通讯网络示意图见图 45-1。

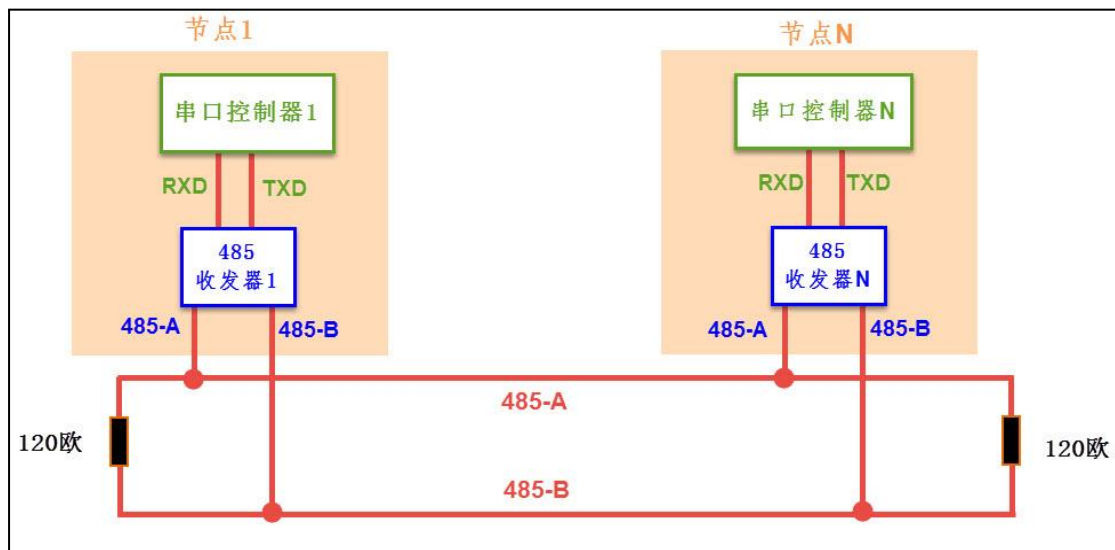


图 45-1 RS-485 通讯网络示意图

对比 CAN 通讯网络，可发现它们的网络结构组成是类似的，每个节点都是由一个通讯控制器和一个收发器组成，在 RS-485 通讯网络中，节点中的串口控制器使用 RX 与 TX 信号线连接到收发器上，而收发器通过差分线连接到网络总线，串口控制器与收发器之间一般使用 TTL 信号传输，收发器与总线则使用差分信号来传输。发送数据时，串口控制器的 TX 信号经过收发器转换成差分信号传输到总线上，而接收数据时，收发器把总线上的差分信号转化成 TTL 信号通过 RX 引脚传输到串口控制器中。

RS-485 通讯网络的最大传输距离可达 1200 米，总线上可挂载 128 个通讯节点，而由于 RS-485 网络只有一对差分信号线，它使用差分信号来表达逻辑，当 AB 两线间的电压差为-6V~-2V 时表示逻辑 1，当电压差为+2V~+6V 表示逻辑 0，在同一时刻只能表达一个信号，所以它的通讯是半双工形式的，它与 RS-232 通讯协议的特性对比见图 45-1。

表 45-1 RS-232/422/485 标准对比

通讯标准	信号线	通讯方向	电平标准	通讯距离	通讯节点数
------	-----	------	------	------	-------

RS-485 与 RS-232 的差异只体现在物理层上，它们的协议层是相同的，也是使用串口数据包的形式传输数据。而由于 RS-485 具有强大的组网功能，人们在基础协议之上还制定了 MODBUS 协议，被广泛应用在工业控制网络中。此处说的基础协议是指前面串口章节中讲解的，仅封装了基本数据包格式的协议(基于数据位)，而 MODBUS 协议是使用基本数据包组合成通讯帧格式的高层应用协议(基于数据包或字节)。感兴趣的读者可查找 MODBUS 协议的相关资料了解。

由于 RS-485 与 RS-232 的协议层没有区别，进行通讯时，我们同样使用 STM32 的 USART 外设作为通讯节点中的串口控制器，再外接一个 RS-485 收发器芯片把 USART 外设的 TTL 电平信号转化成 RS-485 的差分信号即可。

45.2 RS-485—双机通讯实验

本小节演示如何使用 STM32 的 USART 控制器与 MAX485 收发器，在两个设备之间使用 RS-485 协议进行通讯，本实验中使用了两个实验板，无法像 CAN 实验那样使用回环测试(把 STM32 USART 外设的 TXD 引脚使用杜邦线连接到 RXD 引脚可进行自收发测试，不过这样的通讯不经过 RS-485 收发器，跟普通 TTL 串口实验没有区别)，本教程主要以“USART—485 通讯”工程进行讲解。

45.2.1 硬件设计

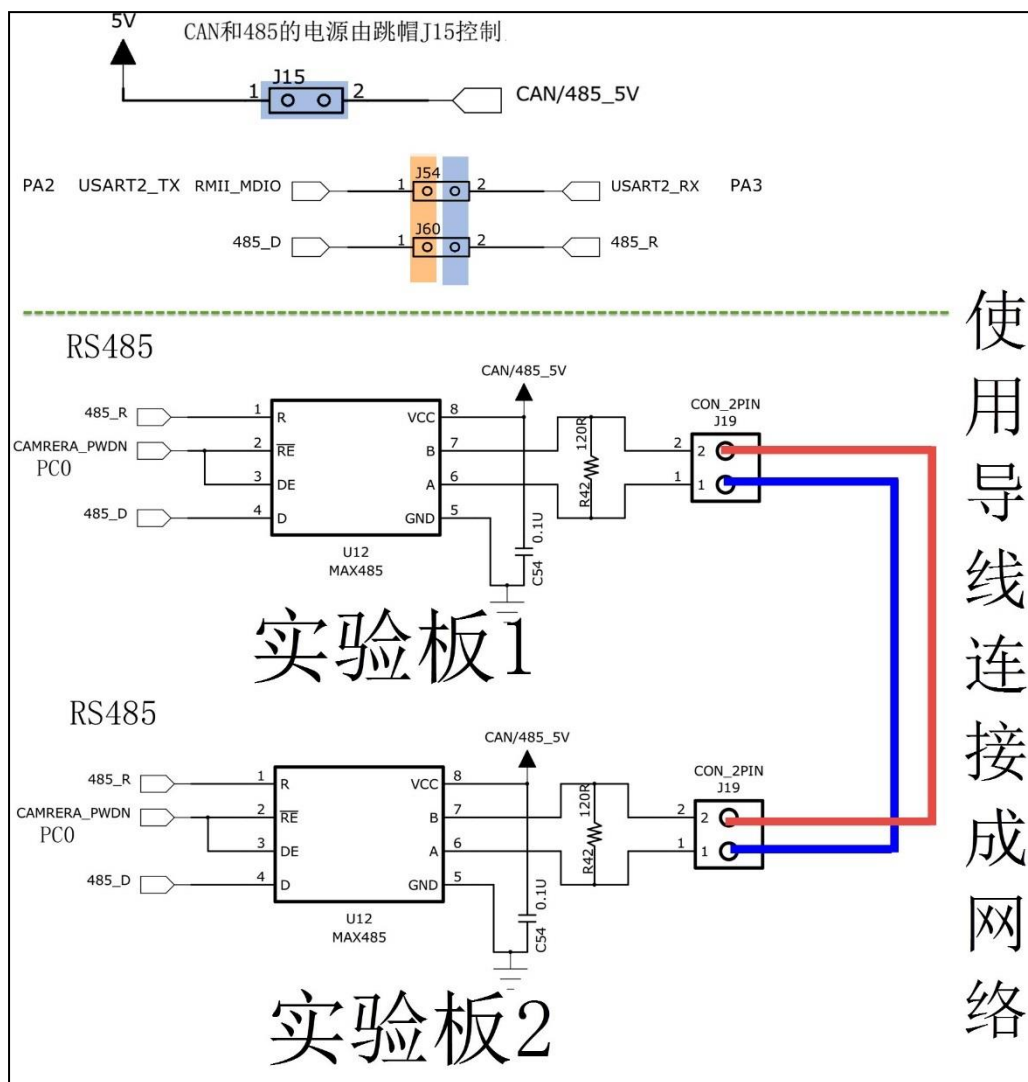


图 45-2 双 CAN 通讯实验硬件连接图

图 44-16 中的是两个实验板的硬件连接。在单个实验板中，作为串口控制器的 STM32 从 USART 外设引出 TX 和 RX 两个引脚与 RS-485 收发器 MAX485 相连，收发器使用它的 A 和 B 引脚连接到 RS-485 总线网络中。为了方便使用，我们每个实验板引出的 A 和 B 之间都连接了 1 个 120 欧的电阻作为 RS-485 总线的端电阻，所以要注意如果您要把实验板作为一个普通节点连接到现有的 RS-485 总线时，是不应添加该电阻的！

由于 485 只能以半双工的形式工作，所以需要切换状态，MAX485 芯片中有“RE”和“DE”两个引脚，用于控制 485 芯片的收发工作状态的，当 RE 引脚为低电平时，485 芯片处于接收状态，当 DE 引脚为高电平时芯片处于发送状态。实验板中使用了 STM32 的 PC0 直接连接到这两个引脚上，所以通过控制 PC0 的输出电平即可控制 485 的收发状态，在本开发板中，PC0 引脚与摄像头使用的引脚共用了，所以使用 485 时不要同时驱动摄像头。

还要注意的，为防止干扰，平时我们默认是不给 485 收发器供电的，使用 485 的时候一定要把 485 接线端子旁边的“C/4-5V”排针使用跳线帽与“5V”排针连接起来进行供电；而又由于实验板的 RS-232 与 RS-485 通讯实验都使用 STM32 的同一个 USART 外设及收发引脚，实验时注意必须要把 STM32 的“PA2 引脚”与 MAX485 的“485_D”及“PA3”与“485_R”使用跳线帽连接起来(这些信号都在 485 接线端子旁边的排针上)。

要实现通讯，我们还要使用导线把实验板引出的 A 和 B 两条总线连接起来，才能构成完整的网络。实验板之间 A 与 A 连接，B 与 B 连接即可。

45.2.2 软件设计

为了使工程更加有条理，我们把 RS485 控制相关的代码独立分开存储，方便以后移植。在“串口实验”之上新建“bsp_485.c”及“bsp_485.h”文件，这些文件也可根据您的喜好命名，它们不属于 STM32 标准库的内容，是由我们自己根据应用需要编写的。这个实验的底层 STM32 驱动与串口控制区别不大，上层实验功能上与 CAN 实验类似。

1. 编程要点

- (1) 初始化 485 通讯使用的 USART 外设及相关引脚；
- (2) 编写控制 MAX485 芯片进行收发数据的函数；
- (3) 编写测试程序，收发数据。

2. 代码分析

485 硬件相关宏定义

我们把 485 硬件相关的配置都以宏的形式定义到“bsp_485.h”文件中，见代码清单 25-2。

代码清单 45-1 485 硬件配置相关的宏(bsp_485.h 文件)

```
1 /*USART 号、时钟、波特率*/
2 #define RS485_USART                USART2
3 #define RS485_USART_CLK             RCC_APB1Periph_USART2
4 #define RS485_USART_BAUDRATE       115200
5
6 /*RX 引脚*/
7 #define RS485_USART_RX_GPIO_PORT   GPIOA
8 #define RS485_USART_RX_GPIO_CLK   RCC_AHB1Periph_GPIOA
9 #define RS485_USART_RX_PIN         GPIO_Pin_3
10 #define RS485_USART_RX_AF          GPIO_AF_USART2
11 #define RS485_USART_RX_SOURCE      GPIO_PinSource3
12
13 /*TX 引脚*/
14 #define RS485_USART_TX_GPIO_PORT   GPIOA
15 #define RS485_USART_TX_GPIO_CLK   RCC_AHB1Periph_GPIOA
16 #define RS485_USART_TX_PIN         GPIO_Pin_2
17 #define RS485_USART_TX_AF          GPIO_AF_USART2
18 #define RS485_USART_TX_SOURCE      GPIO_PinSource2
19
20 /*485 收发控制引脚*/
```

```
21 #define RS485_RE_GPIO_PORT      GPIOC
22 #define RS485_RE_GPIO_CLK      RCC_AHB1Periph_GPIOC
23 #define RS485_RE_PIN            GPIO_Pin_0
24
25 /*中断相关*/
26 #define RS485_INT_IRQ            USART2_IRQn
27 #define RS485_IRQHandler        USART2_IRQHandler
```

以上代码根据硬件连接，把与 485 通讯使用的 USART 外设号、引脚号、引脚源以及复用功能映射都以宏封装起来，并且定义了接收中断的中断向量和中断服务函数，我们通过中断来获知接收数据。

初始化 485 的 USART 配置

利用上面的宏，编写 485 的 USART 初始化函数，见代码清单 25-3。

代码清单 45-2 RS485 的初始化函数(bsp_485.c 文件)

```
1
2 /*
3  * 函数名: RS485_Config
4  * 描述   : USART GPIO 配置,工作模式配置
5  * 输入   : 无
6  * 输出   : 无
7  * 调用   : 外部调用
8  */
9 void RS485_Config(void)
10 {
11     GPIO_InitTypeDef GPIO_InitStructure;
12     USART_InitTypeDef USART_InitStructure;
13
14     /* 配置 USART 时钟 */
15     RCC_AHB1PeriphClockCmd(RS485_USART_RX_GPIO_CLK|
16                             RS485_USART_TX_GPIO_CLK|
17                             RS485_RE_GPIO_CLK, ENABLE);
18     RCC_APB1PeriphClockCmd(RS485_USART_CLK, ENABLE);
19
20     /* TX 引脚源 */
21     GPIO_PinAFConfig(RS485_USART_RX_GPIO_PORT, RS485_USART_RX_SOURCE, RS485_USART_RX_AF);
22
23     /* RX 引脚源 */
24     GPIO_PinAFConfig(RS485_USART_TX_GPIO_PORT, RS485_USART_TX_SOURCE, RS485_USART_TX_AF);
25
26     /* USART GPIO 配置 */
27     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
28     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
29     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
30     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
31
32     /*TX*/
33     GPIO_InitStructure.GPIO_Pin = RS485_USART_TX_PIN;
34     GPIO_Init(RS485_USART_TX_GPIO_PORT, &GPIO_InitStructure);
35
36     /*RX */
37     GPIO_InitStructure.GPIO_Pin = RS485_USART_RX_PIN;
38     GPIO_Init(RS485_USART_RX_GPIO_PORT, &GPIO_InitStructure);
39
40     /* 485 收发控制管脚 */
41     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
42     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
43     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
44     GPIO_InitStructure.GPIO_Pin = RS485_RE_PIN ;
```

```
45     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
46     GPIO_Init(RS485_RE_GPIO_PORT, &GPIO_InitStructure);
47
48     /* USART 模式配置 */
49     USART_InitStructure.USART_BaudRate = RS485_USART_BAUDRATE;
50     USART_InitStructure.USART_WordLength = USART_WordLength_8b;
51     USART_InitStructure.USART_StopBits = USART_StopBits_1;
52     USART_InitStructure.USART_Parity = USART_Parity_No ;
53     USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
54     USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
55
56     USART_Init(RS485_USART, &USART_InitStructure);
57     /*使能 USART*/
58     USART_Cmd(RS485_USART, ENABLE);
59
60     /*配置中断优先级*/
61     NVIC_Configuration();
62     /* 使能串口接收中断 */
63     USART_ITConfig(RS485_USART, USART_IT_RXNE, ENABLE);
64
65     /*控制 485 芯片进入接收模式*/
66     GPIO_ResetBits(RS485_RE_GPIO_PORT, RS485_RE_PIN);
67 }
```

与所有使用到 GPIO 的外设一样，都要先把使用到的 GPIO 引脚模式初始化，配置好复用功能，其中用于控制 MAX485 芯片的收发状态的引脚被初始化成普通推挽输出模式，以便手动控制它的电平输出，切换状态。485 使用到的 USART 也需要配置好波特率、有效字长、停止位及校验位等基本参数，在通讯中，两个 485 节点的串口参数应一致，否则会导致通讯解包错误。在实验中还使能了串口的接收中断功能，当检测到新的数据时，进入中断服务函数中获取数据。

使用中断接收数据

接下来我们编写在 USART 中断服务函数中接收数据的相关过程，见代码清单 25-4，其中的 bsp_RS485_IRQHandler 函数直接被 bsp_stm32f4xx_it.c 文件的 USART 中断服务函数调用，不在此列出。

代码清单 45-3 中断接收数据的过程(bsp_485.c 文件)

```
1 //中断缓存串口数据
2 #define     UART_BUFF_SIZE     1024
3 volatile    uint16_t uart_p = 0;
4 uint8_t     uart_buff[UART_BUFF_SIZE];
5
6 void bsp_RS485_IRQHandler(void)
7 {
8     if (uart_p < UART_BUFF_SIZE) {
9         if (USART_GetITStatus(RS485_USART, USART_IT_RXNE) != RESET) {
10             uart_buff[uart_p] = USART_ReceiveData(RS485_USART);
11             uart_p++;
12
13             USART_ClearITPendingBit(RS485_USART, USART_IT_RXNE);
14         }
15     } else {
16         USART_ClearITPendingBit(RS485_USART, USART_IT_RXNE);
17     }
18 }
19
20 //获取接收到的数据和长度
21 char *get_rebuff(uint16_t *len)
22 {
23     *len = uart_p;
```



```
24     return (char *)&uart_buff;
25 }
26
27 //清空缓冲区
28 void clean_rebuff(void)
29 {
30     uint16_t i=UART_BUFF_SIZE+1;
31     uart_p = 0;
32     while (i)
33         uart_buff[--i]=0;
34 }
```

这个数据接收过程主要思路是使用了接收缓冲区，当 USART 有新的数据引起中断时，调用库函数 USART_ReceiveData 把新数据读取到缓冲区数组 uart_buff 中，其中 get_rebuff 函数可以用于获缓冲区中有效数据的长度，而 clean_rebuff 函数可以用于对缓冲区整体清 0，这些函数配合使用，实现了简单的串口接收缓冲机制。这部分串口数据接收的过程跟 485 收发器无关，是串口协议通用的。

切换收发状态

在前面我们了解到 RS-485 是半双工通讯协议，发送数据和接收数据需要分时进行，所以需要经常切换收发状态。而 MAX485 收发器根据其“RE”和“DE”引脚的外部电平信号切换收发状态，所以控制与其相连的 STM32 普通 IO 电平即可控制收发，为简便起见，我们把收发状态切换定义成了宏，见代码清单 25-5。

代码清单 45-4 切换收发状态(bsp_485.h 文件)

```
1  /// 简单的延时
2  static void RS485_delay(__IO u32 nCount)
3  {
4      for (; nCount != 0; nCount--);
5  }
6
7  /*控制收发引脚*/
8  //进入接收模式,必须要有延时等待 485 处理完数据
9  #define RS485_RX_EN()  RS485_delay(1000);\
10                          GPIO_ResetBits(RS485_RE_GPIO_PORT,RS485_RE_PIN);\
11                          RS485_delay(1000);
12  //进入发送模式,必须要有延时等待 485 处理完数据
13  #define RS485_TX_EN()  RS485_delay(1000);\
14                          GPIO_SetBits(RS485_RE_GPIO_PORT,RS485_RE_PIN);\
15                          RS485_delay(1000);
16
```

这两个宏中，主要是在控制电平输出前后加了一小段时间延时，这是为了给 MAX485 芯片预留响应时间，因为 STM32 的引脚状态电平变换后，MAX485 芯片可能存在响应延时。例如，当 STM32 控制自己的引脚电平输出高电平(控制成发送状态)，然后立即通过 TX 信号线发送数据给 MAX485 芯片，而 MAX485 芯片由于状态不能马上切换，会导致丢失了部分 STM32 传送过来的数据，造成错误。

发送数据

STM32 使用 485 发送数据的过程也与普通的 USART 发送数据过程差不多，我们定义了一个 RS485_SendByte 函数来发送一个字节的数据内容，见代码清单 25-6。

代码清单 45-5 发送数据(bsp_485.c 文件)

```
1
2 /***** 发送一个字符 *****/
3 //使用单字节数据发送前要使能发送引脚, 发送后要使能接收引脚。
4 void RS485_SendByte( uint8_t ch )
5 {
6     /* 发送一个字节数据到 USART1 */
7     USART_SendData(RS485_USART, ch);
8     /* 等待发送完毕 */
9     while (USART_GetFlagStatus(RS485_USART, USART_FLAG_TXE) == RESET);
10
11 }
12
```

上述代码中就是直接调用了 STM32 库函数 USART_SendData 把要发送的数据写入到 USART 的数据寄存器, 然后检查标志位等待发送完成。

在调用 RS485_SendByte 函数前, 需要先使用前面提到的切换收发状态宏, 把 MAX485 切换到发送模式, STM32 发出的数据才能正常传输到 485 网络总线上, 当发送完数据的时候, 应重新把 MAX485 切换回接收模式, 以便获取网络总线上的数据。

3. main 函数

最后我们来阅读 main 函数, 了解整个通讯过程, 见代码清单 25-14。这个 main 函数的整体设计思路是, 实验板检测自身的按键状态, 若按键被按下, 则通过 485 发送 256 个测试数据到网络总线上, 若自身接收到总线上的 256 个数据, 则把这些数据作为调试信息打印到电脑端。所以, 如果把这样的程序分别应用到 485 总线上的两个通讯节点时, 就可以通过按键控制互相发送数据了。

代码清单 45-6 main 函数

```
1
2 /**
3  * @brief 主函数
4  * @param 无
5  * @retval 无
6  */
7 int main(void)
8 {
9
10     char *pbuf;
11     uint16_t len;
12
13     LED_GPIO_Config();
14
15     /*初始化 USART1*/
16     Debug_USART_Config();
17
18     /*初始化 485 使用的串口, 使用中断模式接收*/
19     RS485_Config();
20
21     LED_BLUE;
22
23     Key_GPIO_Config();
24
25     printf("\r\n 欢迎使用野火 STM32 F407 开发板.\r\n");
26     printf("\r\n 野火 F429 485 通讯实验例程\r\n");
27
```

```
28     printf("\r\n 实验步骤: \r\n");
29
30     printf("\r\n 1.使用导线连接好两个 485 通讯设备\r\n");
31     printf("\r\n 2.使用跳线帽连接好:5v --- C/4-5V,485-D --- PA2,485-R ---PA3 \r\n");
32     printf("\r\n 3.若使用两个野火开发板进行实验,给两个开发板都下载本程序即可.\r\n");
33     printf("\r\n 4.准备好后,按下其中一个开发板的 KEY1 键,会使用 485 向外发送 0-255 的数字 \r\n");
34     printf("\r\n 5.若开发板的 485 接收到 256 个字节数据,会把数据以 16 进制形式打印出来. \r\n");
35
36     while (1) {
37         /*按一次按键发送一次数据*/
38         if ( Key_Scan(KEY1_GPIO_PORT,KEY1_PIN) == KEY_ON) {
39             uint16_t i;
40             LED_BLUE;
41             //切换到发送状态
42             RS485_TX_EN();
43
44             for (i=0; i<=0xff; i++) {
45                 RS485_SendByte(i);    //发送数据
46             }
47
48             /*加短暂延时,保证 485 发送数据完毕*/
49             Delay(0xFFFF);
50             RS485_RX_EN();//切换回接收状态
51
52             LED_GREEN;
53             printf("\r\n 发送数据成功! \r\n"); //使用调试串口打印调试信息到终端
54
55         } else {
56             LED_BLUE;
57
58             pbuf = get_rebuff(&len);
59             if (len>=256) {
60                 LED_GREEN;
61                 printf("\r\n 接收到长度为%d 的数据\r\n",len);
62                 RS485_DEBUG_ARRAY((uint8_t*)pbuf,len);
63                 clean_rebuff();
64             }
65         }
66     }
67 }
```

在 main 函数中,首先初始化了 LED、按键以及调试使用的串口,再调用前面分析的 RS485_Config 函数初始化了 RS-485 通讯使用的串口工作模式。

初始化后 485 就进入了接收模式,当接收到数据的时候会进入中断并把数据存储到接收缓冲数组中,我们在 main 函数的 while 循环中(else 部分)调用 get_rebuff 来查看该缓冲区的状态,若接收到 256 个数据就把这些数据通过调试串口打印到电脑端,然后清空缓冲区。

在 while 循环中,还检测了按键的状态,若按键被按下,就把 MAX485 芯片切换到发送状态并调用 RS485_SendByte 函数发送测试数据 0x00-0xFF,发送完毕后切换回接收状态以检测总线的数据。

45.2.3 下载验证

下载验证这个 485 通讯实验需要您有两个实验板,操作步骤如下:

- (1) 按照“硬件设计”小节中的图例连接两个板子的 485 总线;
- (2) 使用跳线帽连接:485_R<--->PA3、485_D<--->PA2、C/4-5V<--->5V;
- (3) 用 USB 线使实验板“USB TO UART”接口跟电脑连接起来,在电脑端打开串口调试助手,编译本章配套的程序,并给两个板子都下载该程序,然后复位。

- (4) 复位后在串口调试助手应看到 485 测试的调试信息，按一下其中一个实验板上的 KEY1 按键，另一个实验板会接收到报文，在串口调试助手可以看到相应的发送和接收的信息。

